

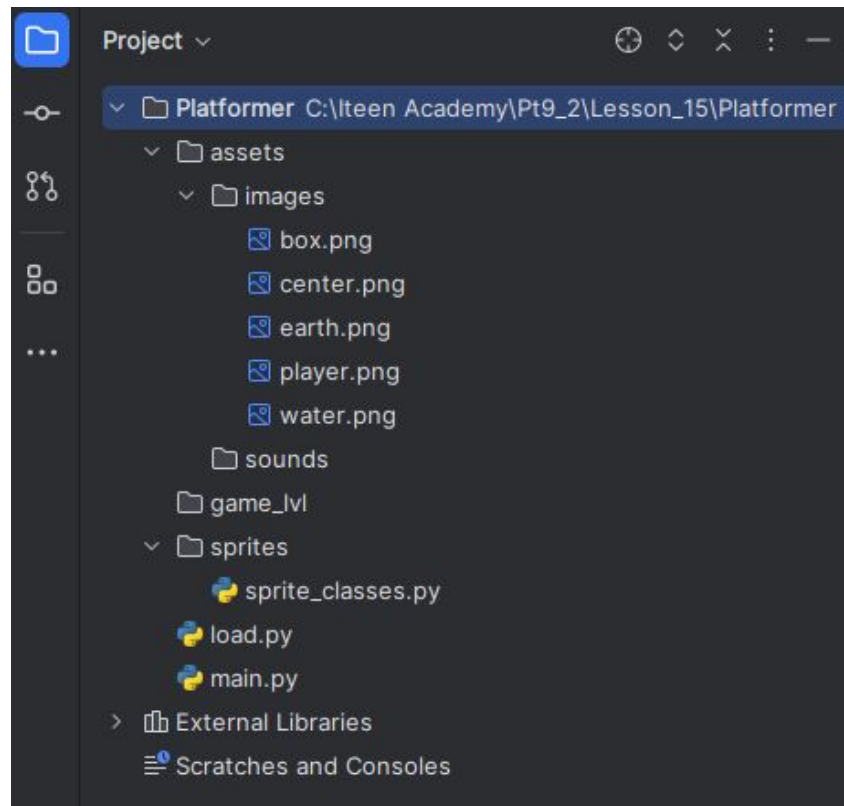
Игра платформер

Создание карты. Игровые классы на основе тайлов

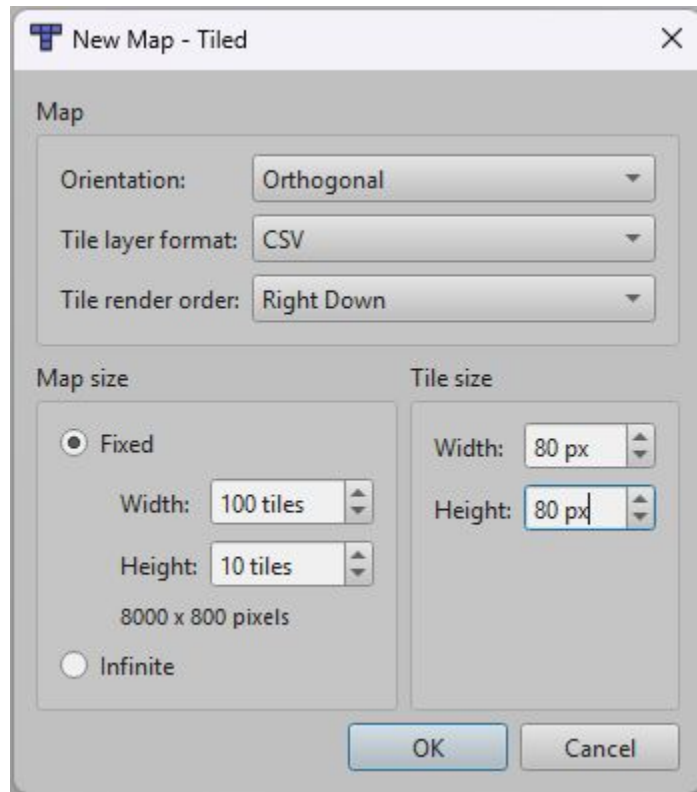
Какой будет игра



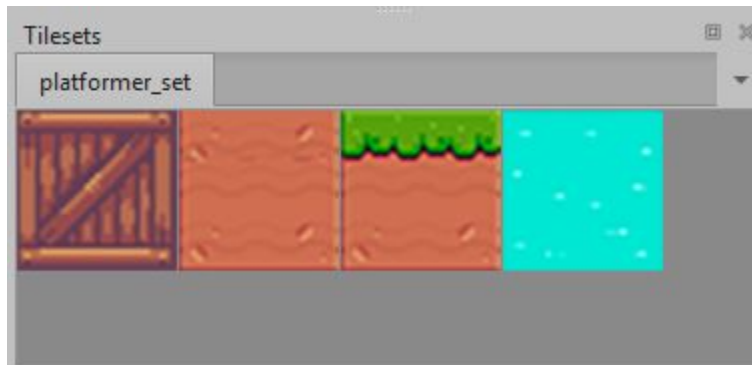
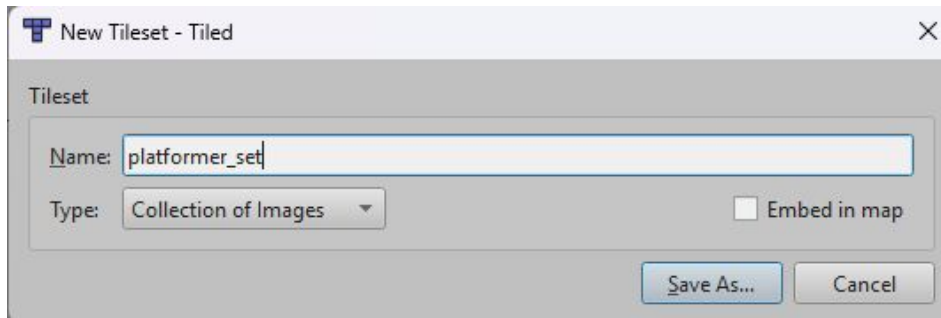
Создаем структуру проекта



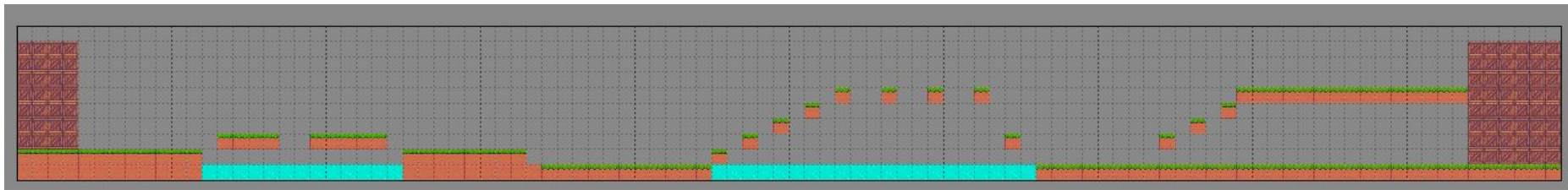
Создаем модель карты (100 x 10) уровня в Tiled



Создаем модель карты (100 x 10) уровня в Tiled

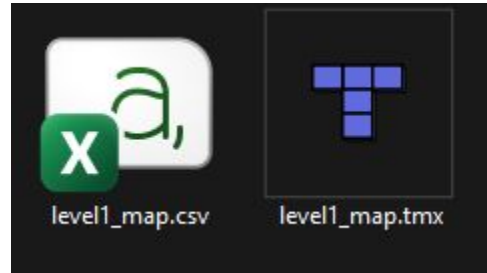


Создаем модель карты (100 x 10) уровня в Tiled



Сохраняем карту

1. Сохраняем карту в файл level1_map.tmx.
2. Экспортируем карту в файл level1_map.csv.



Загрузка файлов с изображениями в модуле load.py

```
import pygame

player_image =
pygame.image.load('assets/images/player.png').convert_alpha()

box_image =
pygame.image.load('assets/images/box.png').convert_alpha()

...
```

Скелет проекта

```
import pygame

import sys

import random

from sprites.sprite_classes import *


GREY = (210, 210, 210)

WIDTH = 1200

HEIGHT = 800

FPS = 60
```

Скелет проекта

```
pygame.init()
```

```
window = pygame.display.set_mode((WIDTH, HEIGHT))
```

```
pygame.display.set_caption("Platformer")
```

```
clock = pygame.time.Clock()
```

```
from load import * # работает только после display.set_mode()
```

Скелет проекта

```
while True:

    clock.tick(FPS)

    for event in pygame.event.get():

        if event.type == pygame.QUIT:

            pygame.quit()

            sys.exit()

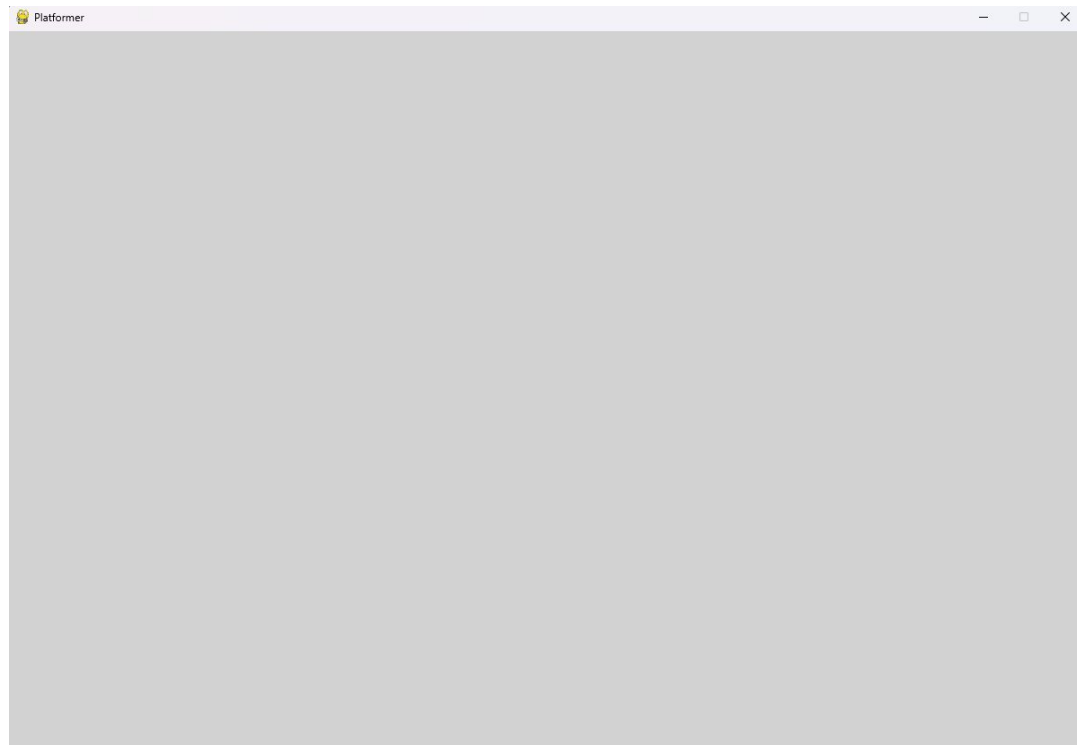
    window.fill(GREY)

    pygame.display.update()
```



Контрольная точка

После запуска проекта на экран
выводится пустое окно.



Создаем классы для каждого блока

```
import pygame
```

```
class Box(pygame.sprite.Sprite):
```

```
    def __init__(self, image, pos):  
        super().__init__()   
        self.image = image  
        self.rect = self.image.get_rect()  
        self.rect.x = pos[0]  
        self.rect.y = pos[1]
```

```
1  import pygame  
2  
3  > class Box(pygame.sprite.Sprite):...  
10  
11 > class Ground(pygame.sprite.Sprite):...  
18  
19 > class Sand(pygame.sprite.Sprite):...  
26  
27 > class Water(pygame.sprite.Sprite):...  
34  
35 > class Player(pygame.sprite.Sprite):...  
42
```

Создаем группы спрайтов

Создание групп спрайтов прописываем в функции `restart()`.

```
def restart() :
```

```
    global box_group, ground_group,
```

```
           sand_group, water_group, player_group
```

```
    box_group = pygame.sprite.Group()
```

```
    ground_group = pygame.sprite.Group()
```

```
    sand_group = pygame.sprite.Group()
```

```
    water_group = pygame.sprite.Group()
```

```
    player_group = pygame.sprite.Group()
```

Рендеринг и обновление спрайтов

```
def lvlGame():  
    global box_group, ground_group,  
        sand_group, water_group, player_group  
    box_group.draw(window)  
  
    ...  
  
    box_group.update()  
  
    ...  
  
    pygame.display.update() # переносим обновление экрана из  
                             игрового цикла
```

Функция отрисовки карты

читаем csv файл и заносим данные в двумерный массив

```
def drawMap(mapFile):
```

```
    game_map = []
```

```
    with open(mapFile, 'r') as file:
```

```
        for i in range(?):
```

```
            game_map.append(file.readline().replace('\n', '').split(','))
```

Функция отрисовки карты

```
def drawMap(mapFile):
```

```
    ...
```

```
    pos = [0, 0]
```

```
    for i in range(?):
```

```
        pos[1] = i * ?
```

```
        for j in range(?):
```

```
            pos[0] = j * ?
```

```
            if game_map[i][j] == '0':
```

```
                # создаем объект соответствующего класса
```

```
                # добавляем объект к соответствующей группе
```

Создаем спрайт игрока

В функции `restart()` создаем спрайт игрока.

```
player = Player(player_image, (?, ?))  
player_group.add(player)
```

Генерируем игровой уровень

```
restart()
```

```
drawMap(?)
```

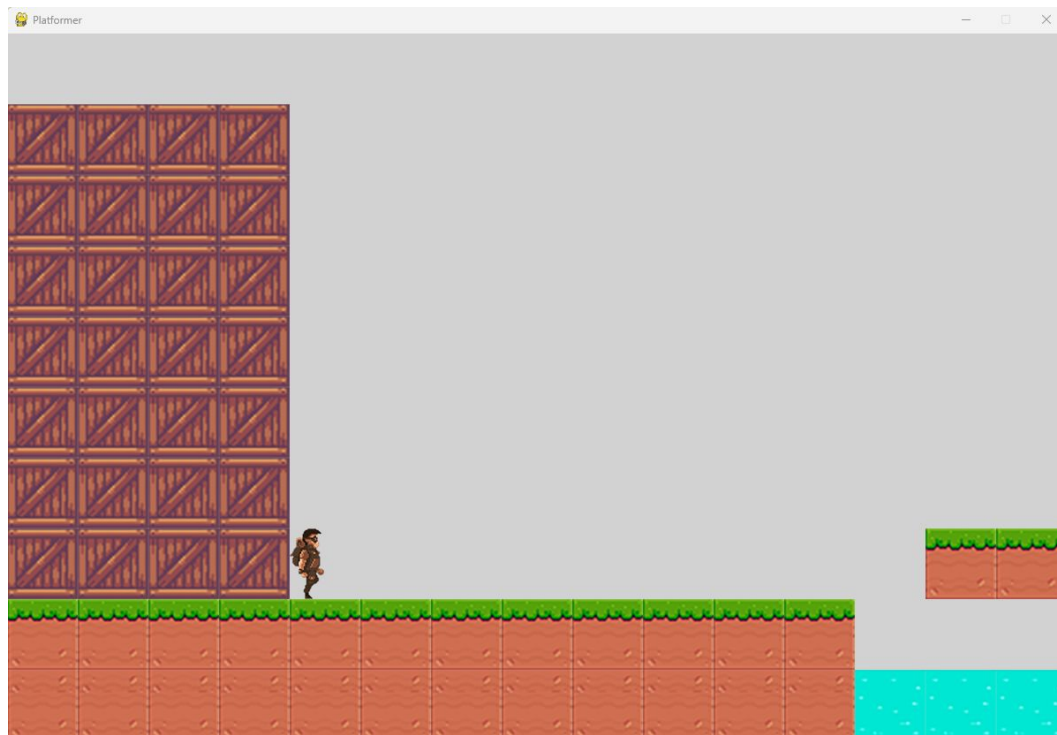
```
while True:
```

```
    ...
```

```
    lvlGame()
```

✓ Контрольная точка

После запуска проекта на экран выводится карта со статичными объектами.



Поведение объектов

Так как карта шире экрана, то для создания эффекта скроллинга будем перемещать блоки в сторону, противоположную движению игрока.

Создаем группу для хранения блоков, которые будут перемещаться:

```
scroll_group = pygame.sprite.Group()
```

Добавляем в группу блоки при создании:

```
if game_map[i][j] == '0':  
    box = Box(box_image, (pos[0], pos[1]))  
    box_group.add(box)  
    scroll_group.add(box)
```

Перемещение игрока

Когда игрок достигает края экрана, он остается на месте, а блоки перемещаются относительно игрока.

В конструктор класса игрока добавьте переменную-параметр `self.speed = 5`

Создайте метод `update()` для игрока:

```
def update(self):  
    key = pygame.key.get_pressed()  
    if key[pygame.K_RIGHT]:  
        self.rect.x += self.speed  
        if self.rect.right > 1000:  
            self.rect.right = 1000  
            scroll_group.update(-self.speed)
```

Методы update() для блоков

```
def update(self, step):  
    self.rect.x += step
```

Обновите параметры методам update() в функции lvlGame() главного модуля.

Подумайте, какое значение задать параметру внутри функции.

Пропишите код для управления движением игрока влево.

Для поворота изображения игрока используйте метод

```
pygame.transform.flip(image, flip_x, flip_y)
```

flip_x, flip_y принимают логические значения

✓ Контрольная точка

Игрок управляется клавишами со стрелками влево, вправо.

При достижении игроком края окна карта движется.

